

Introduction

The MPI library provides a set of communication functions between processes that can be run on one or several machines. Several MPI distributions exist that support different languages like C/C++, Fortran or Python. In this practical session, the C/C++ version of MPI will be used through the OpenMPI distribution (see installation note on Arche). The compilation with MPI requires a *meta*-compiler, usually named `mpicc` and `mpic++`. This *meta*-compiler makes use of a standard compiler (typically `gcc`).

Simple compilation command is as follows : `mpicc source.c -o executable`

However, additional options can be added (warnings, debugging, optimisation,...).

As for the OpenMP session, a `CMakeLists.txt` file is provided to compile automatically through the `cmake` and `make` tools.

In addition, it is required to install a MPI runtime to have the specific execution command `mpirun`. The OpenMPI distribution is suggested, but any other MPI distribution should fit the purpose.

The execution of a MPI program requires to start the different processes on a given list of machines (possibly a single one). This is done by the specific command `mpirun`. The most important options are (see manual for a detailed list of options) :

- `-n <int>` : total number of instances of the program to execute (nb of processes)
- `-host <host:P,...>` : list of machines with their numbers of local processes (P)
- `-machinefile <file>` : text file containing the list of machines

When using multiple machines, direct connections between them must be done without explicit password typing. This can be achieved with an adequate configuration of `ssh` for example.

In this practical session, only one machine will be used. The `localhost` is a specific name representing the machine from which MPI is started. For example, the command to execute four instances (processes) of `myprog` on your machine is :

```
mpirun -n 4 -host localhost:4 ./myprog
```

In such a case, the `machinefile` option is not required.

Additional debugging information are available by including `debug_MPI.h` in the source files. To use it, the function `Debug_Init` must be called *after* `MPI_Init()`. Also, each MPI function must be encapsulated in the macro `Debug_MPI( ... )`. For example :

```
Debug_MPI( MPI_Comm_rank(MPI_COMM_WORLD, &numP) );
```

Exercise 1 : Communication system starting

The first program in source file `creation.c` simply starts the MPI communication system and displays some information about each process.

- a) Execute the program with 1, 2, 4 and 8 processes. What do you observe?
- b) Execute the program without using the `mpirun` command. What can you deduce?
- c) Execute the program several times with at least 4 processes. What about the displays order?
- d) Add point-to-point synchronous communications to the program so that processes are displayed in ascending order of their IDs.

Exercise 2 : Approximation of π

The next program (`pi.c`) computes an approximation of π with a single process.

a) Execute the program with different numbers of trapezoids (option `-n` followed by an integer). Note the evolution of the error over π and of the computation times.

b) Modify the program to distribute the trapezoids over the set of processes. The global set of trapezoids must form a partition of the interval $[0:1]$. Execute the program with as many processes as the number of cores available on your machine. What do you observe concerning the computation time? And concerning the partial results obtained by the processes?

c) Add the required communications to merge all the partial results on process 0.

Exercise 3 : Counting letters occurrences in a text

The last program, in source file `occurrences.c`, generates a pseudo-random text of given size (option `-n`) and counts the number of occurrences of the letters without distinction between lowercase and uppercase. In this program, a sequential version and a parallel version are included. However, the parallel version is not operational and must be completed.

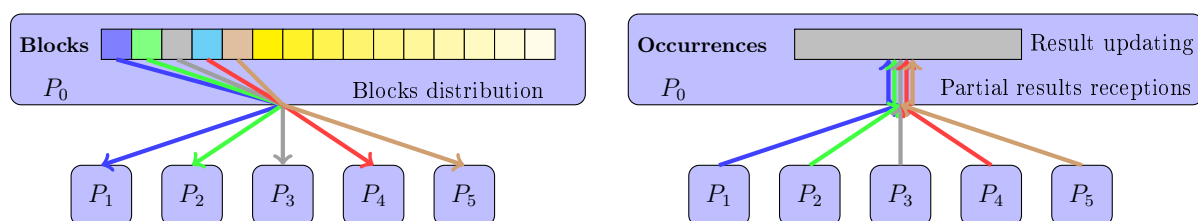
a) Execute the program with different text sizes. What do you observe about the letters frequencies when the text size increases?

In the parallel function `count_occs_par()`, a *master-workers* scheme is implemented. The principle is that one process (the *master*) distributes small pieces of the work to the other processes (*workers*) that do the work and return their results to the master. In our case, a piece of work is defined by a block of the initial text with given size (option `-b`).

On the master side, a first distribution is done to the workers (one block each). Then, the master waits for the results from the workers. At each result reception, the master updates the final result (occurrences) and sends to the corresponding worker another block of text. This is repeated until there is no more block to send (all the text has been processed). In this last case, the master sends empty text messages to the workers to signal the termination of the work.

On the workers side, there is a conditional loop in which a text block is received, locally processed, and the result is sent back to the master. The loop stops when the received message corresponds to the termination signal (empty message).

At the end, the global result is available only on the master process. The figure below illustrates the global strategy with six processes and P_0 as the master.



b) Complete the empty parts of the parallel function. It may be easier to begin with the workers (simple scheme : reception-processing-sending).